Programación de GPUs
Práctica evaluable: Detección de carriles con CUDA

1. Introducción

En esta práctica se desarrollará una versión acelerada mediante GPU de un algoritmo de detección de carriles en imágenes.

Este tipo de algoritmos se utilizan en sistemas de asistencia a la conducción (*Advanced Driver Assistance Systems*, ADAS) y constituyen un ejemplo representativo de aplicación del procesamiento de imágenes en tiempo real.

El objetivo del algoritmo es detectar las líneas de carril presentes en una imagen capturada desde la perspectiva de un vehículo.

Figura 1: Ejemplo del resultado final esperado del algoritmo.

El código proporcionado incluye una implementación completamente funcional del algoritmo en CPU. El objetivo de esta práctica consiste en trasladar las partes más costosas computacionalmente a GPU utilizando CUDA.

Además de la implementación CUDA, se deberá realizar un pequeño estudio experimental del rendimiento obtenido.

2. Código proporcionado

La práctica incluye una implementación completa del algoritmo en CPU.

Los ficheros más relevantes son:

- `main.c`
- `routinesCPU.c`

- `routinesGPU.h`
- `png_io.c`

El programa principal se ejecuta de la siguiente forma:

```
./exec imagen.png c
./exec imagen.png g
```

El primer argumento indica la imagen de entrada. El segundo argumento selecciona la versión a ejecutar:

- `c`: ejecución en CPU
- `g`: ejecución en GPU

El objetivo de la práctica consiste en implementar la función:

```
lane_assist_GPU(...)
```

de forma que reproduzca el comportamiento de la versión CPU.

3. Descripción general del algoritmo

El algoritmo completo sigue una serie de etapas bien definidas.

1. Conversión de la imagen RGB a escala de grises
2. Detección de bordes
3. Transformada de Hough
4. Extracción de líneas
5. Dibujado del resultado

Estas etapas se encuentran implementadas en la función:

```
lane_assist_CPU(...)
```

del fichero `routinesCPU.c`. Esta función constituye un buen punto de partida para entender el funcionamiento global del algoritmo.

4. Conversión a escala de grises

Las imágenes de entrada se encuentran en formato RGB.

Para simplificar el procesamiento, se convierten a escala de grises mediante la combinación ponderada:

$$I = 0.2989R + 0.5870G + 0.1140B$$

Esta operación se encuentra implementada en la función:

```
image_RGB2BW(...)
```

5. Detección de bordes

La detección de bordes se realiza mediante una implementación simplificada del detector de Canny.

Figura 2: Ejemplo de detección de bordes mediante el detector de Canny.

5.1. Reducción de ruido

Antes de calcular el gradiente se aplica un filtro de suavizado basado en convolución.
La convolución se define como:

$$I'(x,y) = \sum_{i=-k}^{k} \sum_{j=-k}^{k} w(i,j) I(x+i, y+j)$$

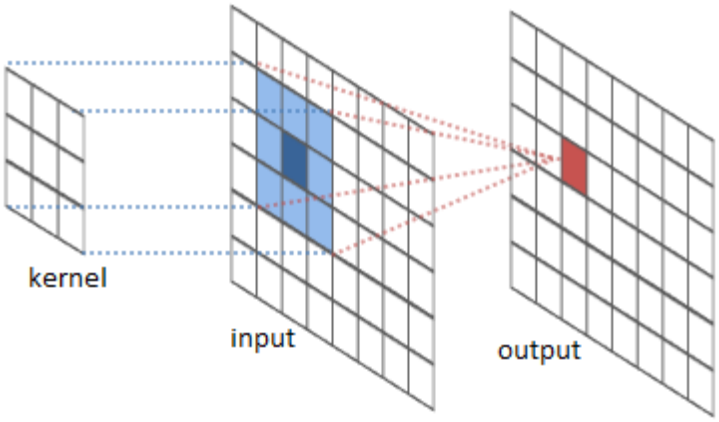

Figura 3: Ejemplo conceptual de convolución aplicada a una imagen.

El filtro utilizado en el código es:

$$\frac{1}{159} \begin{bmatrix} 2 & 4 & 5 & 4 & 2 \\ 4 & 9 & 12 & 9 & 4 \\ 5 & 12 & 15 & 12 & 5 \\ 4 & 9 & 12 & 9 & 4 \\ 2 & 4 & 5 & 4 & 2 \end{bmatrix}$$

5.2. Gradiente

El gradiente de la imagen se calcula mediante dos filtros de convolución:

$$G_x = \begin{bmatrix} 1 & 2 & 0 & -2 & -1 \\ 4 & 8 & 0 & -8 & -4 \\ 6 & 12 & 0 & -12 & -6 \\ 4 & 8 & 0 & -8 & -4 \\ 1 & 2 & 0 & -2 & -1 \end{bmatrix}$$

$$G_y = \begin{bmatrix} -1 & -4 & -6 & -4 & -1 \\ -2 & -8 & -12 & -8 & -2 \\ 0 & 0 & 0 & 0 & 0 \\ 2 & 8 & 12 & 8 & 2 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix}$$

La magnitud del gradiente se calcula como:

$$G = \sqrt{G_x^2 + G_y^2}$$

6. Transformada de Hough

Una vez detectados los bordes se buscan líneas rectas utilizando la transformada de Hough. La ecuación de la recta se expresa como:

$$\rho = x\cos(\theta) + y\sin(\theta)$$

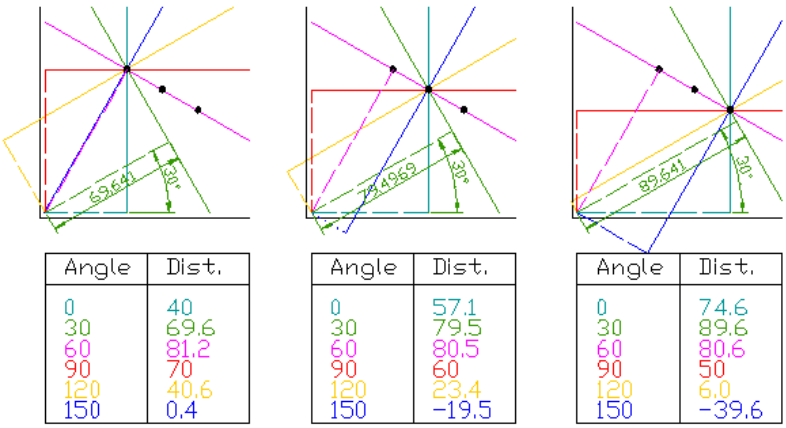

Figura 4: Proceso de votación de la transformada de Hough.

Cada píxel de borde vota en un acumulador para múltiples valores de θ.

```
for each edge pixel
    for theta = 0..179
        rho = x*cos(theta) + y*sin(theta)
        accumulator[rho][theta]++
```

6.1. Condiciones de carrera

En GPU múltiples hilos pueden intentar actualizar simultáneamente la misma celda del acumulador.

Para evitar condiciones de carrera se utilizan operaciones atómicas:

```
atomicAdd(&accumulator[index],1);
```

7. Paralelización con CUDA

Las imágenes son estructuras bidimensionales, lo que facilita su paralelización.
Cada hilo puede procesar un píxel:

```
int x = blockIdx.x * blockDim.x + threadIdx.x;
int y = blockIdx.y * blockDim.y + threadIdx.y;
```

8. Posibles optimizaciones

8.1. Memoria constante

Las tablas trigonométricas pueden almacenarse en memoria constante:

```
__constant__ float sin_table[180];
__constant__ float cos_table[180];
```

8.2. Memoria compartida

Las operaciones de convolución pueden beneficiarse del uso de memoria compartida para reducir accesos a memoria global.

9. Estudio de rendimiento

Además de implementar el algoritmo en GPU se deberá realizar un estudio experimental del rendimiento.
El speedup se define como:

$$Speedup = \frac{T_{CPU}}{T_{GPU}}$$

donde T_{CPU} es el tiempo de ejecución de la versión en CPU y T_{GPU} es el tiempo total de ejecución de la versión en GPU.

9.1. Tiempos a medir

En la versión GPU se deberán reportar los siguientes tiempos:

- Tiempo total de ejecución en GPU (incluyendo transferencias de memoria).
- Tiempo total de ejecución de los kernels CUDA.
- Tiempo dedicado a transferencias de memoria entre CPU y GPU (Host $\leftrightarrow$ Device).

9.2. Comparación CPU vs GPU

Resolución	Tiempo CPU	Tiempo GPU total	Speedup

9.3. Desglose de tiempos en GPU

Resolución	Transferencias H2D/D2H	Kernels	Total GPU

9.4. Metodología experimental

Para que los resultados sean reproducibles se deberán seguir las siguientes recomendaciones:

- Ejecutar cada experimento varias veces (por ejemplo entre 10 y 30 repeticiones).
- Descartar una o varias ejecuciones iniciales de *warm-up*.
- Reportar el tiempo medio de ejecución.
- Utilizar las mismas imágenes de entrada para todas las mediciones.
- Indicar en la memoria el hardware utilizado (modelo de CPU y GPU).

Como recomendación, se sugiere incluir al menos una gráfica que muestre la evolución del tiempo de ejecución o del speedup en función del tamaño de la imagen.

10. Descripción del entregable

La práctica se evaluará en dos partes.

10.1. Implementación CUDA (40 %)

Se evaluará:

- Correctitud de la implementación
- Funcionamiento del programa
- Generación del resultado correcto

10.2. Memoria técnica (60 %)

La memoria deberá incluir:

- Estrategia de paralelización
- Descripción del uso de operaciones atómicas
- Resultados experimentales
- Discusión de optimizaciones

La memoria debe describir la implementación realizada por el estudiante. Respuestas genéricas que no hagan referencia al código entregado o a los resultados experimentales obtenidos se considerarán insuficientes.

11. Estrategia de implementación y errores frecuentes

11.1. Estrategia recomendada para portar el código CPU

Se recomienda trasladar el algoritmo a GPU de forma progresiva, siguiendo una estrategia incremental.

1. Comprender el flujo completo del algoritmo en CPU leyendo la función `lane_assist_CPU()` en `routinesCPU.c`.
2. Identificar las partes del algoritmo que pueden paralelizarse fácilmente. En particular:
   - convoluciones de suavizado
   - cálculo del gradiente
   - detección de bordes
   - transformada de Hough
3. Diseñar kernels CUDA que procesen píxeles de la imagen en paralelo.
4. Gestionar correctamente la memoria del dispositivo:
   - reservar memoria con `cudaMalloc`
   - copiar datos con `cudaMemcpy`
   - ejecutar kernels
   - copiar resultados de vuelta al host
5. Implementar el algoritmo de forma incremental y verificar cada etapa.

11.2. Errores frecuentes

Durante el desarrollo de la práctica es habitual encontrarse con algunos errores comunes:

- accesos fuera de rango en las operaciones de convolución
- errores en el cálculo del índice del acumulador de Hough
- olvidar inicializar el acumulador antes de usarlo
- no utilizar operaciones atómicas al actualizar el acumulador
- configuraciones incorrectas de bloques CUDA